Stock Market Price Prediction Trends using AI/ML

By Prithvi Koka

Link to Video:

https://drive.google.com/file/d/1Ywa6vD8hS_9c1X4UxPiPIK4tJJzKh-sD/view?usp=sharing

Problem:

- The stock market is a never ending story of ups and downs.
- There is a way to forecast stocks through looking at historical data.
- Can lead to a lot of financial gain if we can correctly predict if the stock is going up as well as preventing risk in investing in negative trend stocks.

Motivation

- I personally invest in stocks when I can.
- The stock market can cause us to lose a lot of money especially with high risk investments such as options or shorts.
- By using algorithms, I can prevent myself from investing in high risk stocks.
- Can expand this to a bigger market for example stock brokers and advisors.

Existing Related Approaches

- In a A Study on Stock Forecasting Using Deep Learning and Statistical Models Published on February 8th, 2024 on arXiv By Himanshu Gupta and Aditya Jaiswal, they delve into training models using different approaches such as LSTM, Arima, CNN, and RNN in order to predict the stock price the next day.
- Replicate this same exact approach by developing the algorithms for these models from the lessons we learned in class.

My Method

- I use a Stock Market dataset from Kaggle that has stock data up until 2020.
- Kaggle: https://www.kaggle.com/datasets/jacksoncrow/stock-market-dataset
- I want to test out 5 specific stocks. Apple, Amazon, Microsoft, Nvidia, and Google for the model.
- Use code from class as well as developed my own code.

LSTM

```
model = Sequential([
    LSTM(128, return_sequences=True, input_shape=(LOOKBACK, X.shape[2])),
    Dropout(0.2),
    LSTM(64, return_sequences=False),
    Dropout(0.2),
    Dense(32, activation='relu'),
    Dense(1)
])

model.compile(optimizer='adam', loss='mse')
model.summary()
```

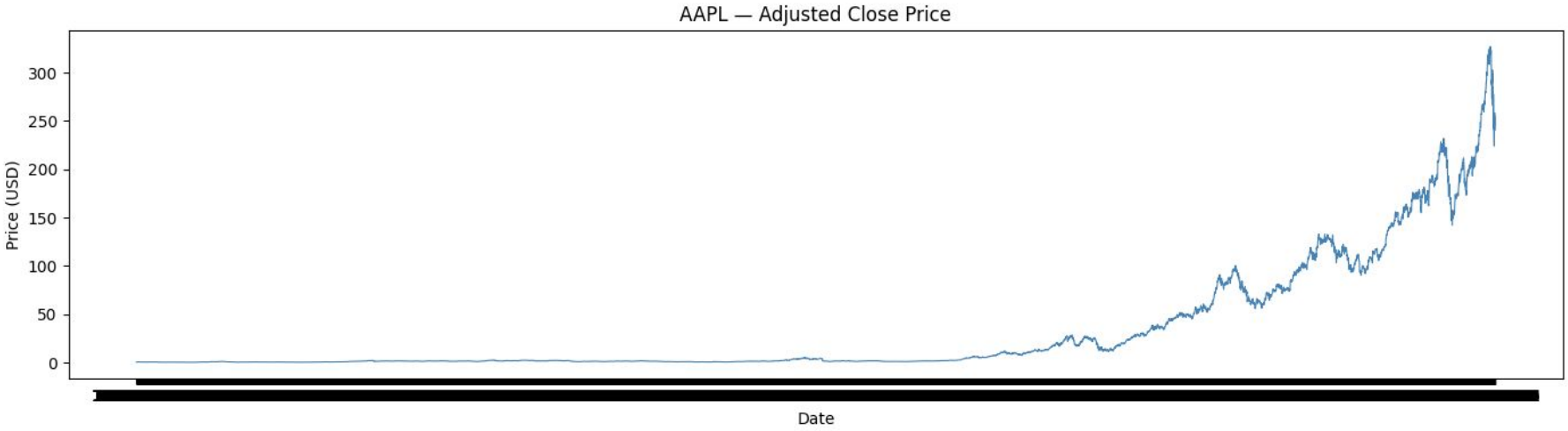
Convolutional Neural Network

```
from tensorflow.keras.layers import Conv1D, MaxPooling1D, Flatten, Dense, Dropout

model1 = Sequential([
    Conv1D(filters=64, kernel_size=3, activation='relu',
           input_shape=(LOOKBACK, X.shape[2])),
    MaxPooling1D(pool_size=2),
    Dropout(0.2),

    Conv1D(filters=128, kernel_size=3, activation='relu'),
    MaxPooling1D(pool_size=2),
    Dropout(0.2),

    Flatten(),
    Dense(64, activation='relu'),
    Dropout(0.2),
    Dense(1)
])

model1.compile(optimizer='adam', loss='mse')
model1.summary()
```

Recurrent Neural Network

```
from tensorflow.keras.layers import SimpleRNN, Dense, Dropout

model2 = Sequential([

    SimpleRNN(128, activation='tanh', return_sequences=True,
              input_shape=(LOOKBACK, X.shape[2])),
    Dropout(0.2),


    SimpleRNN(64, activation='tanh', return_sequences=False),
    Dropout(0.2),


    Dense(32, activation='relu'),
    Dense(1)
])

model2.compile(optimizer='adam', loss='mse')
model2.summary()
```

Results

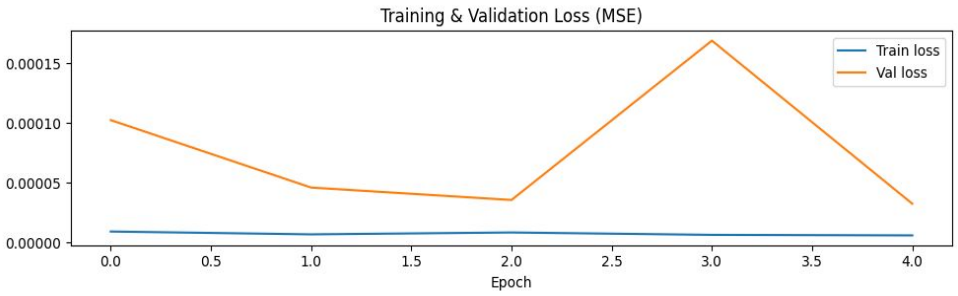

AAPL — Adjusted Close Price
Price (USD)
300
250
200
150
100
50
0
Date

LSTM Results:

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 60, 128)	67,072
dropout (Dropout)	(None, 60, 128)	0
lstm_1 (LSTM)	(None, 64)	49,408
dropout_1 (Dropout)	(None, 64)	0
dense (Dense)	(None, 32)	2,080
dense_1 (Dense)	(None, 1)	33

Total params: 118,593 (463.25 KB)
Trainable params: 118,593 (463.25 KB)
Non-trainable params: 0 (0.00 B)




```
62/62 ━━━━━━━━━━━━━━━━━━━━ 3s 42ms/step
MAE  : $11.00
RMSE : $17.58
MAPE : 6.38%
```

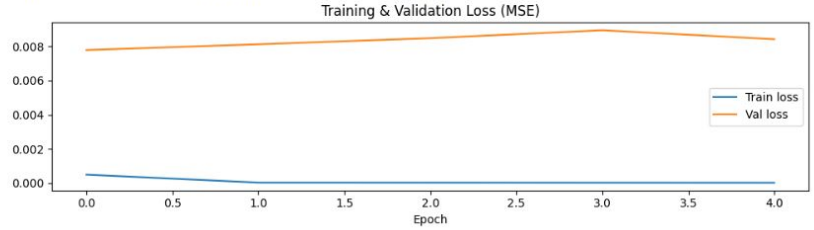
CNN Results:

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 58, 64)	448
max_pooling1d (MaxPooling1D)	(None, 29, 64)	0
dropout_2 (Dropout)	(None, 29, 64)	0
conv1d_1 (Conv1D)	(None, 27, 128)	24,704
max_pooling1d_1 (MaxPooling1D)	(None, 13, 128)	0
dropout_3 (Dropout)	(None, 13, 128)	0
flatten (Flatten)	(None, 1664)	0
dense_2 (Dense)	(None, 64)	106,560
dropout_4 (Dropout)	(None, 64)	0
dense_3 (Dense)	(None, 1)	65

Total params: 131,777 (514.75 KB)
Trainable params: 131,777 (514.75 KB)
Non-trainable params: 0 (0.00 B)

```
Epoch 1/5
222/222 ━━━━━━━━━━━━━━━━━━━━ 5s 13ms/step - loss: 4.9744e-04 - val_loss: 0.0078
Epoch 2/5
222/222 ━━━━━━━━━━━━━━━━━━━━ 4s 20ms/step - loss: 3.4188e-05 - val_loss: 0.0081
Epoch 3/5
222/222 ━━━━━━━━━━━━━━━━━━━━ 4s 16ms/step - loss: 3.1879e-05 - val_loss: 0.0085
Epoch 4/5
222/222 ━━━━━━━━━━━━━━━━━━━━ 4s 12ms/step - loss: 2.7961e-05 - val_loss: 0.0089
Epoch 5/5
222/222 ━━━━━━━━━━━━━━━━━━━━ 3s 12ms/step - loss: 2.5513e-05 - val_loss: 0.0084
```




```
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step
MAE  : $109.82
RMSE : $122.45
MAPE : 79.11%
```

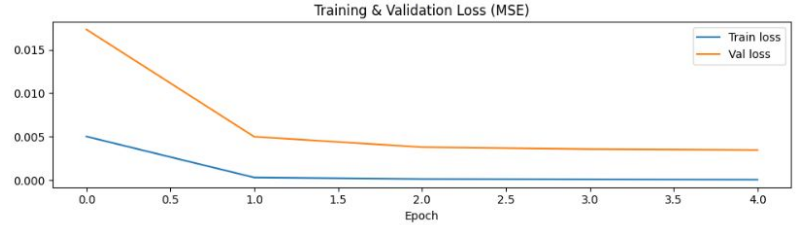
RNN Results:

```
super().__init__(**kwargs)
Model: "sequential_2"
```

Layer (type)	Output Shape	Param #
simple_rnn (SimpleRNN)	(None, 60, 128)	16,768
dropout_5 (Dropout)	(None, 60, 128)	0
simple_rnn_1 (SimpleRNN)	(None, 64)	12,352
dropout_6 (Dropout)	(None, 64)	0
dense_4 (Dense)	(None, 32)	2,080
dense_5 (Dense)	(None, 1)	33

```
Total params: 31,233 (122.00 KB)
Trainable params: 31,233 (122.00 KB)
Non-trainable params: 0 (0.00 B)
```

```
Epoch 1/5
222/222 ━━━━━━━━━━━━━━━━━━━━ 12s 41ms/step - loss: 0.0050 - val_loss: 0.0173
Epoch 2/5
222/222 ━━━━━━━━━━━━━━━━━━━━ 7s 33ms/step - loss: 3.0089e-04 - val_loss: 0.0050
Epoch 3/5
222/222 ━━━━━━━━━━━━━━━━━━━━ 11s 37ms/step - loss: 1.2886e-04 - val_loss: 0.0038
Epoch 4/5
222/222 ━━━━━━━━━━━━━━━━━━━━ 10s 39ms/step - loss: 9.5751e-05 - val_loss: 0.0036
Epoch 5/5
222/222 ━━━━━━━━━━━━━━━━━━━━ 8s 37ms/step - loss: 5.5787e-05 - val_loss: 0.0035
```




```
62/62 ━━━━━━━━━━━━━━━━━━━━ 1s 15ms/step
MAE  : $101.83
RMSE : $116.60
MAPE : 70.95%
```

ARIMA Results TBD

The results for ARIMA are still to be determined because I am still finishing up writing the results for it. Will be done by report.

Conclusion

- Overall, it seems like LSTM is the best because it is designed for sequential data with long term dependencies. The memory gate can let it remember patterns from long time ago.
- CNN seems to come in second because it does a good job at detecting short price patterns, but it struggles with long ones. Lookback might have caused it to seem better than RNN (Shorter time window). Also results are very similar.
- RNN has a vanishing gradient problem so it struggles to remember long term data and gets outperformed by LSTM and CNN.
- ARIMA Results TBD

Bibliography

Citations (MLA):

- “A Study on Stock Forecasting Using Deep Learning and Statistical Models.” Arxiv.org, 2024, arxiv.org/html/2402.06689v1.
- Subasi, Abdulhamit, et al. “Stock Market Prediction Using Machine Learning.” Procedia Computer Science, vol. 194, 2021, pp. 173–179, https://doi.org/10.1016/j.procs.2021.10.071.
- Chakravorty, Amitabh, and Nelly Elsayed. “A Comparative Study of Machine Learning Algorithms for Stock Price Prediction Using Insider Trading Data.” ArXiv.org, 2025, arxiv.org/abs/2502.08728.